



FACULTY OF COMPUTING

UNIVERSITY OF

SRI JAYEWARDENEPURA

CCS2002 | CSE2002 Group Project

Unified Business Management

System

Interim Report

April 2026

Name – R.M.L.G.Rathnadoowa

Group Number – 09

Index Number – FC115252

Registration Number – FC221007

Name of the Supervisor – Mr. K.D.T.Rangana

Table of Contents

1. INTRODUCTION

1.1 Motivation for the project.....	3
1.2 Objectives.....	4
1.3 Scope of the project.....	4

2. ANALYSIS

2.1 Current manual method.....	5
2.2 Drawbacks of existing method.....	5
2.3 Review of similar systems.....	5
2.4 Requirements of the system.....	7
2.4.1 Functional requirements.....	7
2.4.2 Non-Functional requirements.....	8
2.5 Process Model.....	9

3. DESIGN

3.1 Design strategies.....	10
3.2 Justification for the selected design strategy.....	10
3.3 Architectural design.....	11
3.4 Object-Oriented methodical approach.....	11
3.5 High-Level use case diagram for the proposed system.....	12
3.6 ER Diagram for the proposed system.....	12
3.7 Class diagram for the proposed system.....	13
3.8 User interface designing for the system.....	14

4. CONCLUSION

4.1 Introduction.....	15
4.2 Problems Encountered.....	15
4.3 Future Improvements.....	15
4.4 Lessons Learnt.....	15

5. INDEX

1. INTRODUCTION

1.1 Motivation for the project

The idea behind developing UBMS arises from the following motivations:

- **To make the operations smooth**

Manual entry and switching between multiple software tools creates delays, errors, and duplication. UBMS will automate quotation to invoice conversion, real-time stock adjustments and tax calculations and reports.

- **To make workers happy to engage in their tasks**

Currently, employees struggle with outdated systems, leading to stress and low productivity. A centralized and user-friendly platform will minimize repetitive work, improve task clarity and allow fast decision-making through dashboards.

- **To automate the paper based modules**

Invoices, stock sheets, and quotations are currently paper-based. UBMS will digitize these workflows with email integration, digital signature support and document backup in the cloud.

- **To reduce waste**

By automating the system, we can reduce the paper waste, print waste and any other waste that happens in a manual system.

- **To generate the reports easily**

It is very convenient to get updated information from the system and generate required reports and also it saves time.

- **To make the tracking easy**

As the company grows, the system must grow too. UBMS is modular and built to be reused across other businesses with minimal code changes.

- **To make it easy to share data among others**

Considering the existing manual system, it is very easy to share data and the documents among others as all are automated through the new system.

1.2 Objectives

The main goal of this project is to enhance the day-to-day operations of this business. It will replace the legacy system with a smart solution, workflow will be automated and also can maintain up-to-date inventories with minimum manual effort. Moving through this solution, we can ensure data integrity and business continuity through backups and recovery. So the main intention is to build a system that is modular, scalable, and suitable for future adaptations.

1.3 Scope of the project

The Unified Business Management System (UBMS) will serve as an all-in-one platform to manage the business operations of a company importing and selling electrical items. As the business scales, manual methods (like QuickBooks or spreadsheets) have become inefficient, error-prone, and slow. The new system will automate, simplify, and centralize daily operations.

The system will comprise the following core modules:

1. Stock Maintenance Module – Real-time inventory updates, purchase order tracking.
2. Quotation Generator – Issue quotations directly to clients.
3. Invoice Management – Create standard and tax-compliant invoices (including SVAT).
4. Profit Calculation Module – Automates gross and net profit reporting.
5. Tax Calculation Module – Supports VAT, SVAT, and other local tax regulations.
6. User and Role Management – Manage admin and staff access.
7. Payment & Finance Management – Track receivables, payables, and payment status.
8. Data Backup and Recovery – Ensure data is secure and retrievable.

The target users are:

- Sales Executives
- Inventory Managers
- Finance Officers
- Business Owners/Admins

The system is designed to be scalable and flexible to adapt to other businesses beyond the current client. The software is also designed to scale across industries, meaning it can be reused by other companies with minimal customization.

2. ANALYSIS

2.1 Current manual method

At present, the target organization manages its operations using a combination of QuickBooks software and manual record-keeping methods such as spreadsheets and physical documents. Inventory updates are often recorded manually, and there is no real-time synchronization between stock levels and sales transactions. Quotations and invoices are generated separately, sometimes using templates, which increases the chances of inconsistency.

Customer records, supplier details, and payment tracking are handled in a semi-digital manner, leading to scattered data across different systems. Tax calculations, including VAT and SVAT, are performed manually or with limited software support, which increases the risk of calculation errors. Additionally, there is no centralized dashboard to monitor overall business performance, making decision-making slower and less efficient.

Overall, the current method lacks integration, automation, and real-time visibility, making it difficult to manage the growing scale of business operations effectively.

2.2 Drawbacks of existing method

The existing system has several limitations that directly affect operational efficiency and business growth:

- **Lack of Integration:** Different processes such as inventory, invoicing, and finance are not connected, leading to data inconsistency.
- **Time-Consuming Processes:** Manual entry and repetitive tasks slow down daily operations.
- **High Risk of Errors:** Manual calculations for stock and taxes increase the likelihood of human errors.
- **Limited Real-Time Insights:** No centralized dashboard for monitoring stock levels, sales, or financial performance.
- **Poor Scalability:** The current system cannot efficiently handle increasing data volume or business expansion.
- **Data Redundancy:** Duplicate data entries across spreadsheets and documents.
- **Limited Accessibility:** Business owners cannot easily access data remotely or via mobile devices.

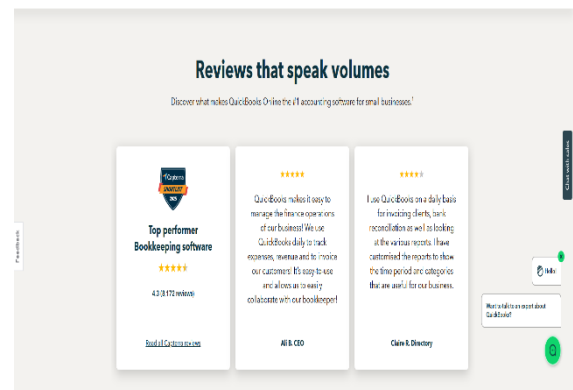
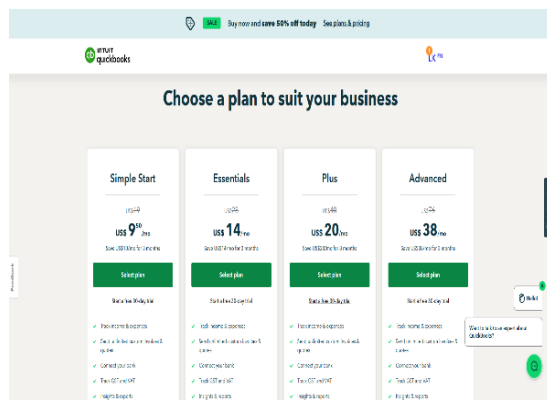
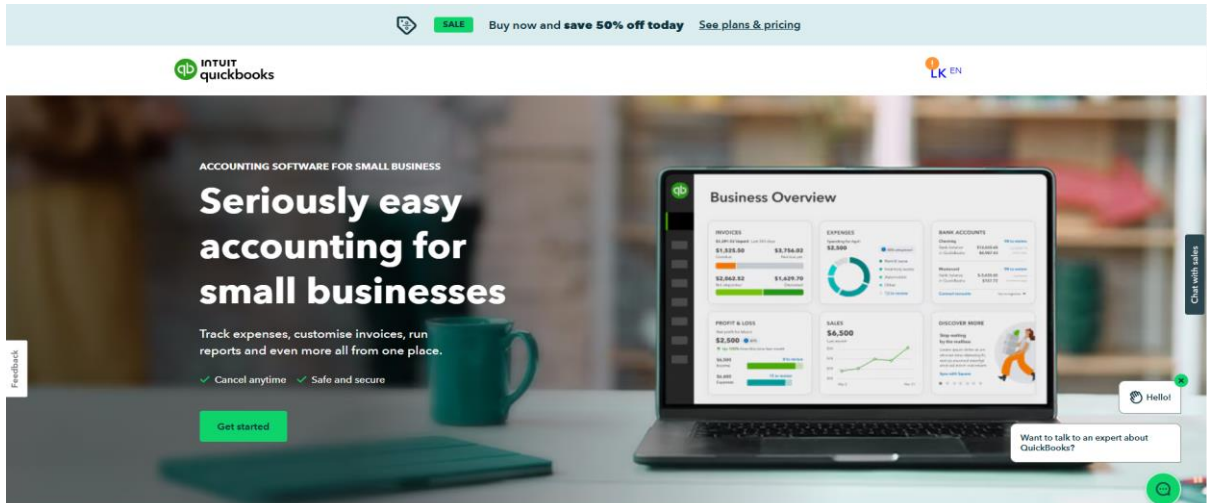
These drawbacks highlight the need for a more automated, integrated, and scalable system like UBMS.

2.3 Review of similar systems

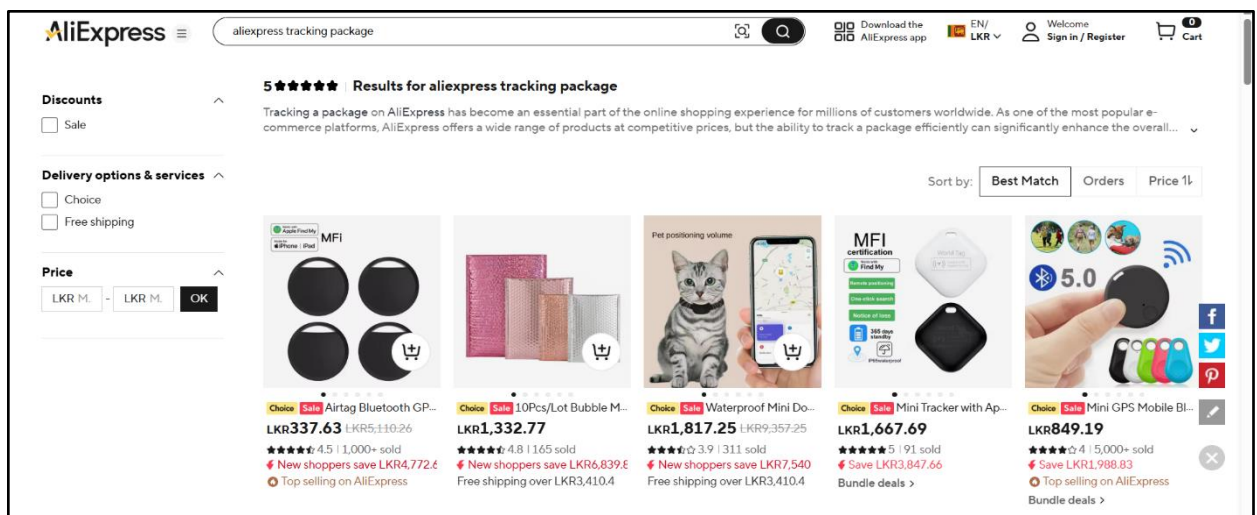
We have identified several similar platforms like our project. The Project Manager and the System Analyst collaboratively with other members observed their features and identified the gaps in the existing systems. Below are the similar platforms we identified.

❖ QuickBooks

QuickBooks is an accounting software developed by Intuit that helps small and medium-sized businesses manage their finances. It offers both cloud-based and on-premises versions to handle tasks like invoicing, expense and inventory tracking, payroll, and financial reporting. By automating many manual tasks, QuickBooks aims to help businesses save time and gain financial insights. This is the currently used software in the target business of our project.



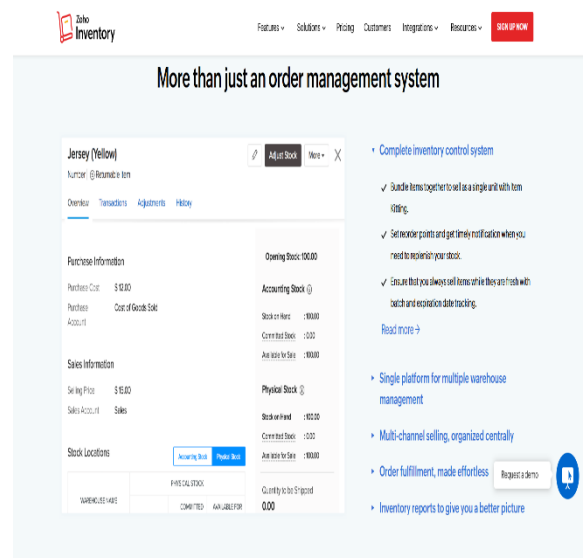
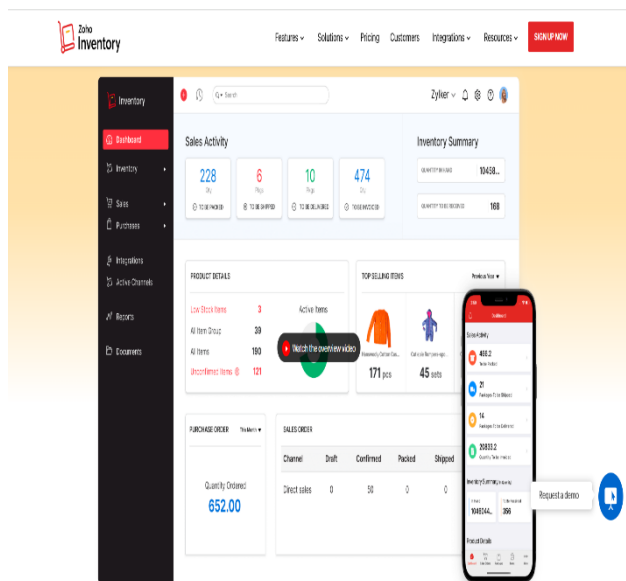
❖ AliExpress



AliExpress is a global e-commerce platform owned by the Alibaba Group that allows international consumers to buy products directly from Chinese manufacturers and sellers. We identified Aliexpress as a similar platform for our project that suits the order tracking feature.

❖ Zoho inventory

Zoho Inventory is a cloud-based software for businesses to manage their inventory, orders, and multi-channel sales from a single platform. It helps businesses track stock, process sales and purchase orders, manage warehouses, and handle shipping and payments. The software includes features like automated alerts for low stock, serial and batch tracking, and real-time reporting to help optimize operations.



2.4 Requirements of the system

2.4.1 Functional requirements

Each requirement defines what the system should do:

Manage Users Features:

- Add/edit/delete users.
- Assign user roles (Admin, Sales, Inventory Manager, Accountant).
- Allow login/logout.
- Password encryption and reset functionality.
- Access restrictions based on roles.

Stock Management Features:

- Add new product items with model, brand, supplier, price, and warranty.

- Real-time stock level updates upon sales, purchase, or return.
- Alert for low stock level (e.g., "Below Reorder Point").
- Generate purchase orders.
- Sync data between dashboard and mobile app.

Quotation and Invoice System Quotation Module:

- Generate customer-specific quotations.
- Include tax rates and validity dates.
- Send as PDF or email directly. Invoice Module:
- Create sales invoices linked to stock.
- Normal VAT/SVAT invoices with correct formatting.
- Calculate total with tax, discounts, and charges.

Payment and Finance Management Features:

- Record payments (full, partial, or credit).
- Show outstanding dues for each customer.
- Filter payment history by date, status, or customer.
- Export monthly payment and sales reports.

Profit and Tax Calculation Features:

- Calculate gross profit per item or invoice.
- Deduct purchase cost, discounts, and taxes to show net profit.
- Support for multiple tax types (VAT, SVAT).
- Generate reports by period, brand, or category.

Backup and Recovery Features:

- Automatic daily database backup.
- Manual backup on-demand.
- Restore functionality to a previous backup point.
- Backup logs and status monitoring.

2.4.2 Non-Functional requirements

Usability

- The user interface should be clean, modern, and easy to navigate.
- Tooltips and error messages should guide users effectively.
- Mobile view must be optimized for dashboard use.

Performance

- Load time for dashboard and search under 2 seconds.
- Handle 1000+ product records without slowing down.

Security

- All passwords encrypted (bcrypt hashing).
- Role-based access control.
- Audit logs for sensitive actions (like price or tax edits).

Reliability

- System uptime target: 99.9%.
- Daily backups and logging to prevent data loss.

Scalability

- Easily add new modules (e.g., supplier returns).
- The system should support additional stores or branches.

Maintainability

- Modular code structure.
- Easily upgradable database schema and UI components.

Portability

- Should work on Chrome, Firefox, Edge, and mobile browsers.
- Compatible with Windows and Linux servers.

2.5 Process Model

2.5.1 Methodology options

➤ **Waterfall method**

The Waterfall model is a linear and sequential software development method where each phase (requirements, design, implementation, testing, deployment, maintenance) must be completed before moving to the next. It follows a one-directional flow with strict documentation and milestone control. This method works well when requirements are fixed and fully understood at the beginning. However, changes are difficult once development starts, and customer feedback is only incorporated at the end.

Not suitable because our project requires flexibility, evolving requirements, and continuous feedback from the business owners, which Waterfall cannot efficiently support.

➤ **V model**

The V-Model is an enhanced form of the Waterfall approach where each development stage has a corresponding testing phase, forming a “V” structure. It emphasizes strict validation and verification at every level, making it highly suitable for error-critical systems such as medical or defence software. The development process is rigid and heavily documentation-driven. Though it improves error control, it still lacks flexibility once requirements are defined. Not suitable because our business software needs adaptive changes, iterative improvements, and fast feedback cycles, which the rigid V-Model methodology does not support.

➤ **Iterative model**

The Iterative model develops software through repeated cycles, improving features based on feedback after each version. Each iteration includes planning, design, coding, and testing, allowing enhancements without building the entire system at once. This method supports evolving requirements and encourages early delivery of functional modules. However, it may lack structured planning across multiple iterations, and managing scope can become difficult

in large teams without a defined framework. Not suitable alone because while it supports iteration, our project needs structured sprint planning, continuous collaboration, and clear team-role distribution — which Agile provides more effectively.

➤ **Spiral model**

The Spiral model combines iterative development with risk analysis and prototyping. Each cycle includes planning, risk assessment, engineering, and evaluation, making it useful for large, high-risk projects. It focuses heavily on identifying and mitigating risks early. However, the continuous risk evaluation makes it complex, time-consuming, and expensive to execute. Not suitable because our system has moderate risk and targeted scope; the Spiral model would introduce unnecessary overhead and delays compared to a faster and more collaborative Agile approach.

2.5.2 Why choose Agile?

As we mentioned in our project proposal and the project SRS, we are using the Agile methodology for our system making process. Since we are new to requirement gathering, we are new to design and development it is better to choose the Agile path for our work. And also the system requirements are still in the changing process so it seems that Agile is the best methodology for our project. Agile is Sprint-based, iterative, flexible and customer-driven so that it is easy to manage the work process for us.

3. DESIGN

3.1 Design strategies

The design of the UBMS system follows a modular and scalable approach to ensure flexibility and maintainability. The system is divided into independent modules such as stock management, invoicing, finance, and user management. Each module is designed to function independently while interacting with others through well-defined APIs.

A layered architecture strategy is adopted, separating the system into presentation (frontend), application logic (backend), and data (database) layers. This separation improves system organization and simplifies future updates.

The system also follows a user-centered design approach, ensuring that the interface is intuitive and easy to use for non-technical users. Additionally, responsive design principles are applied so that the system can be accessed via both desktop and mobile devices. Security and performance considerations are integrated into the design, including role-based access control and optimized data handling mechanisms.

3.2 Justification for the selected design strategy

The chosen design strategy is suitable for this project due to several key reasons.

- ❖ Modularity allows easy addition or modification of features without affecting the entire system.
- ❖ Scalability ensures the system can handle increasing data and users as the business grows.
- ❖ Maintainability is improved through separation of concerns, making debugging and updates easier.
- ❖ Flexibility supports customization for different types of businesses in the future.
- ❖ Performance Efficiency is achieved by optimizing interactions between frontend, backend, and database layers.

Since the system is intended to evolve and expand beyond the initial client, a modular and layered design provides a strong foundation for long-term development.

3.3 Architectural design

The UBMS system is designed using a three-tier architecture consisting of:

1) Presentation Layer (Frontend)

Developed using React.js, HTML, and CSS, this layer provides the user interface through which users interact with the system.

2) Application Layer (Backend)

Implemented using Java and Node.js, this layer handles business logic, API processing, and communication between the frontend and database.

3) Data Layer (Database)

Managed using MongoDB, this layer stores all system data including products, customers, invoices, and transactions.

The system uses RESTful APIs for communication between layers. This architecture ensures clear separation of responsibilities, improved performance, and easier scalability.

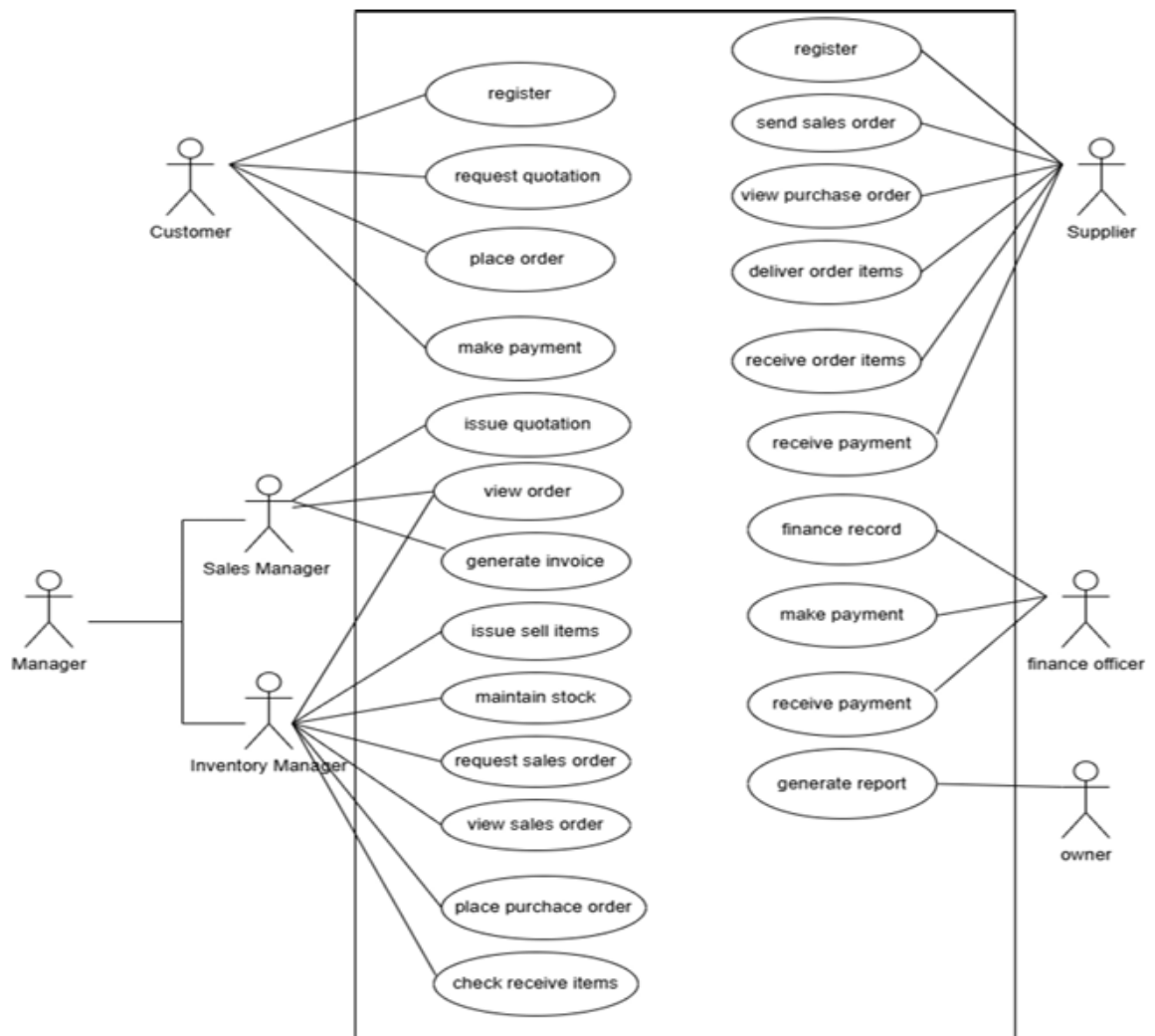
3.4 Object-Oriented methodical approach

The system follows an Object-Oriented Programming (OOP) approach to model real-world business entities as software objects. Key entities such as Product, Customer, Supplier, Invoice, and Payment are represented as classes with attributes and methods.

Encapsulation is used to protect sensitive data, while inheritance allows reuse of common properties among related classes. Polymorphism enables flexible implementation of different business operations.

This approach improves code reusability, maintainability, and scalability, making it suitable for a complex business system like UBMS.

3.5 High-Level use case diagram for the proposed system

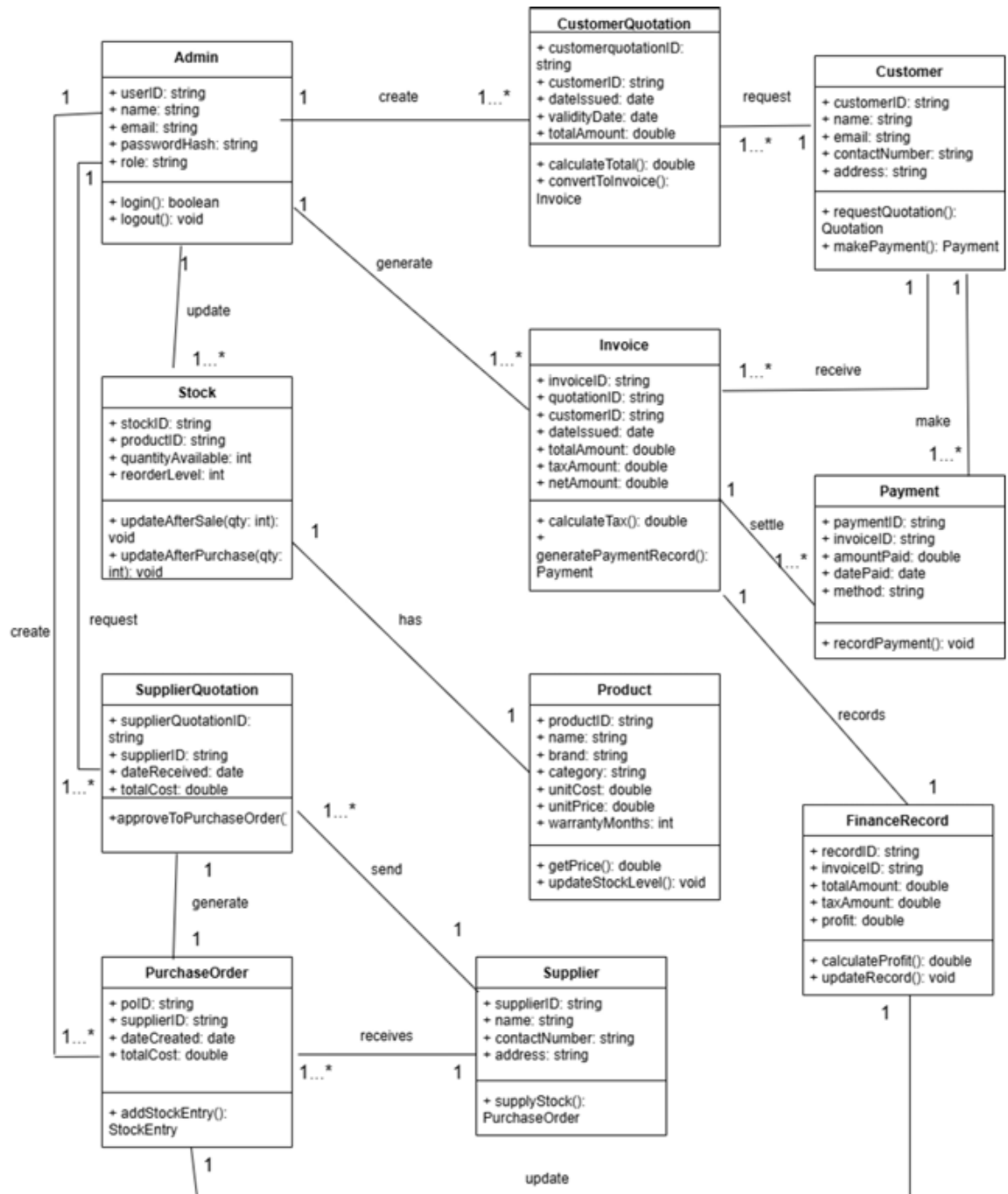


3.6 ER Diagram for the proposed system

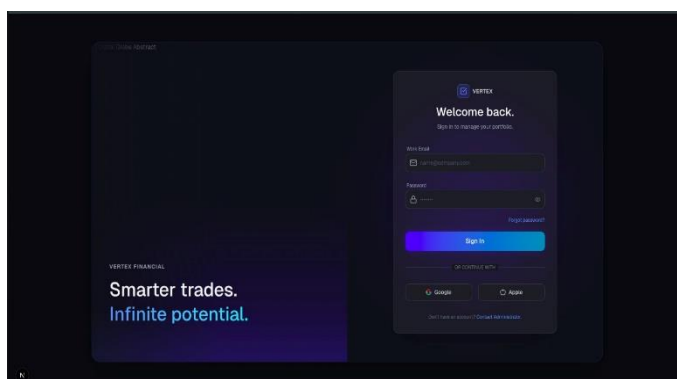
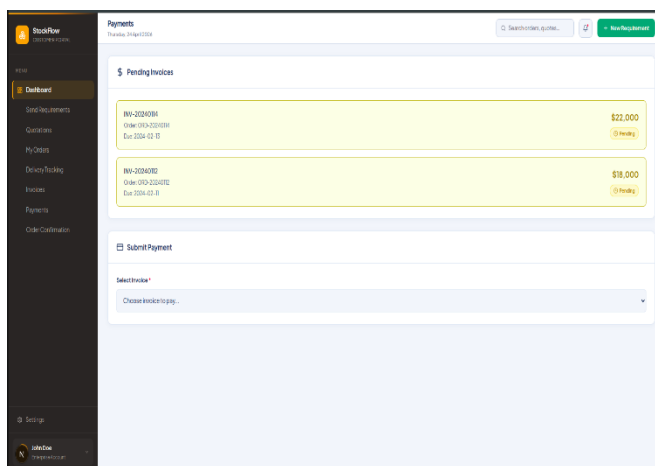
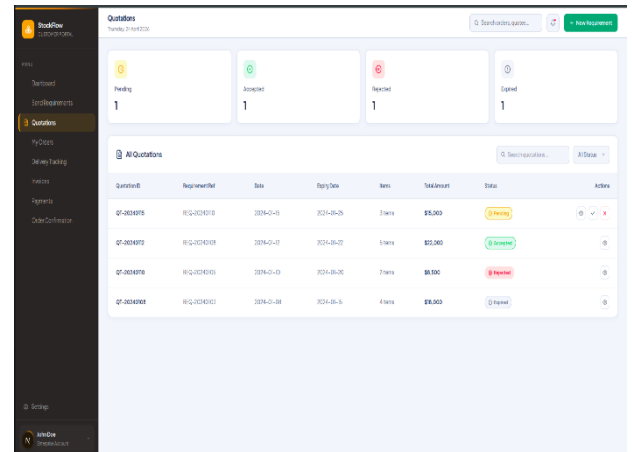
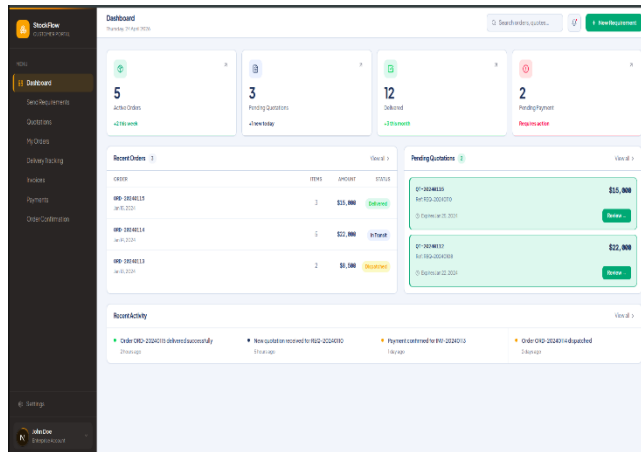
Click here to view the full ER diagram :

<https://drive.google.com/file/d/1SBtQyBM3yOUZ0jWDp73L2Mt4JXCZZ8Y0/view?usp=sharing>

3.7 Class diagram for the proposed system



3.8 User interface designing for the system



4. CONCLUSION

4.1 Introduction

This interim report outlines the progress made in the development of the Unified Business Management System (UBMS). It covers analysis, design decisions, and the current status of the project. The report demonstrates that the project is progressing according to the planned schedule with a clear understanding of system requirements and design structure.

4.2 Problems Encountered

During the initial stages of the project, several challenges were identified:

- Difficulty in clearly understanding all business workflows at the beginning.
- Managing coordination among team members with different responsibilities.
- Learning and integrating multiple technologies such as backend frameworks and database systems.
- Designing a system flexible enough to support multiple business types.
- Time management challenges during requirement gathering and documentation.

These challenges were addressed through team discussions, research, and proper task allocation.

4.3 Future Improvements

The system can be further enhanced with the following features:

- Development of a mobile application for real-time monitoring.
- Integration of AI-based analytics and forecasting.
- Multi-branch and multi-user enterprise support.
- Cloud-based deployment for better accessibility and scalability.
- Advanced reporting and data visualization tools.

4.4 Lessons Learnt

Throughout this phase of the project, several important lessons were learned:

- The importance of clear requirement gathering before development.
- Effective teamwork and communication are critical for project success.
- Proper planning and task distribution improve productivity.
- Agile methodology helps manage changing requirements efficiently.
- Hands-on experience with modern tools and technologies enhances practical knowledge.

These lessons will guide the team in the upcoming development phases to ensure successful project completion.

5. INDEX

Role	Name	Reg.No.
Project Manager	R.M.L.G.Rathnadoowa	FC115252
Data Analyst	R.D.R.P.Ranasinghe	FC115596
Designer	B.A.M.J.Ranasingha	FC115670
Frontend Developer	T.A.K.V.I De Alvis	FC115559
Backend Developer	W.M.A.T.Weerasekara	FC115281
	T.G.K.Pabodha	FC116347
Database Administrator	B.P.U.Mendis	FC115530
Tester	D.T.Paranavithana	FC115738

.....